

Reverse-Engineering the Seals of the Goetia

A Learning Journey Through Computer Vision, Graph Theory, and Statistical Analysis

Ted Hand

<https://github.com/t3dy/goetia-sigil-analysis>

Table of Contents

Introduction

Chapter 1: Seeing the Grid

Chapter 2: Finding the Bones

Chapter 3: Mapping the Intersections

Chapter 4: Detecting Geometry

Chapter 5: Measuring Complexity

Chapter 6: Discovering Families

Chapter 7: Visualizing the Families

Chapter 8: Bridging Text and Image

Chapter 9: Building the Database

Chapter 10: Thinking in Graphs

Chapter 11: Learning to Generate

Chapter 12: The Shape of Sound

Chapter 13: Shared Vocabulary

Chapter 14: Reading the Sequence

Chapter 15: Preparing for Interactivity

Chapter 16: The Correction

Conclusion

Introduction

The Lesser Key of Solomon contains seventy-two seals, each attributed to a different demon of the Goetia. These sigils have been reproduced for centuries in manuscripts and printed grimoires, passed from scribe to scribe with varying degrees of fidelity. To most observers they appear as abstract tangles of lines and circles, but to a computational eye they are something else entirely: a structured visual language waiting to be decoded.

This project began with a single question. Are these sigils arbitrary, or do they follow rules? The resemblance between certain seals and early circuit diagrams had been noted before, but no one had subjected the entire corpus to systematic computational analysis. What followed was an eighteen-script pipeline that progressively dissected these images using techniques drawn from computer vision, topology, graph theory, signal processing, and statistical learning. Each script solved a specific problem and in doing so taught me a new technique or deepened my understanding of one I thought I already knew.

What follows is the story of that pipeline, told chapter by chapter, problem by problem. It is not a tutorial and it is not a reference manual. It is a record of learning through doing, of reaching for a tool because a question demanded it, and of discovering things about both the tools and the sigils that I did not expect.

01_segment_sigils.py



The first problem was deceptively simple: given a single image containing all seventy-two seals arranged in a grid, extract each one as a separate file. A human can see the grid instantly, but a computer sees only a matrix of pixel intensities with no inherent notion of rows, columns, or boundaries.

I reached for OpenCV, the workhorse library of computer vision, because it provides the morphological operations needed to bridge the gap between raw pixels and meaningful regions. The approach works in three stages. First, the image is converted to binary by thresholding: every pixel above a brightness cutoff becomes white, everything else becomes black. This isolates the ink from the parchment. Second, morphological dilation expands each ink blob outward by a few pixels, causing nearby strokes

to merge into connected regions. The trick here is calibrating the dilation strength. Too little and a single seal fragments into dozens of disconnected pieces. Too much and adjacent seals merge into one. I swept through dilation levels from one to seven and scored each by how many contiguous regions it produced, looking for the count closest to seventy-two.

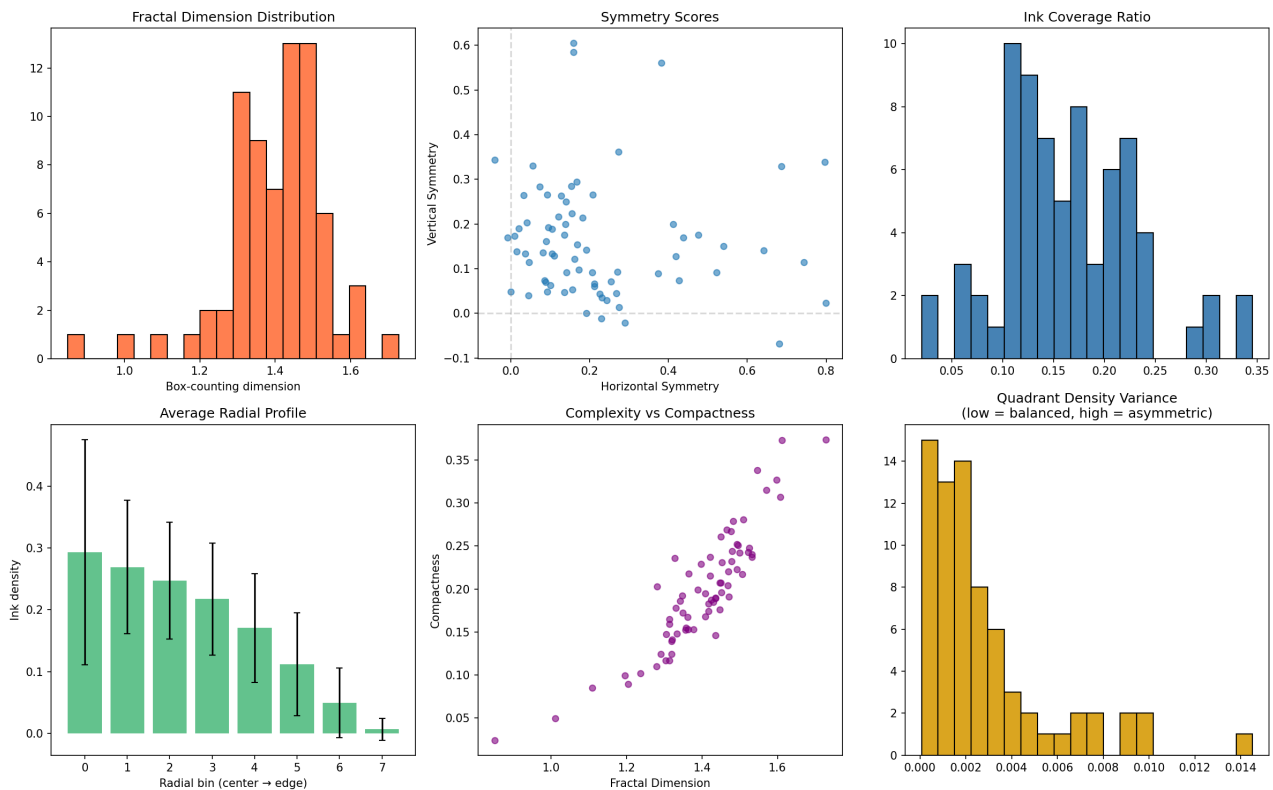
The third stage uses contour detection to find the bounding rectangle of each merged region. OpenCV's `findContours` traces the outer boundary of each blob and returns its coordinates, from which a bounding box can be computed. Sorting these boxes into reading order required a spatial clustering step: group boxes by their vertical position to identify rows, then sort left-to-right within each row.

What I learned was that morphological operations are not just image processing primitives but spatial reasoning tools. Dilation is really a question about connectivity: how close do two ink marks need to be before we consider them part of the same object? That question recurs throughout the entire pipeline in different forms.

CHAPTER 2

Finding the Bones

02_skeleton_analysis.py



Distribution of key features across all 72 sigils, including fractal dimension, ink ratio, and symmetry scores.

With individual sigils in hand, the next question was structural: what is the underlying topology of each seal? A sigil's visual weight comes from its stroke thickness, but its identity comes from its connectivity, the way lines branch, cross, and terminate. To reveal that connectivity I needed to reduce each image to a one-pixel-wide skeleton while preserving its topological properties.

The Zhang-Suen thinning algorithm, available through scikit-image's `skeletonize` function, does exactly this. It works by iteratively peeling away boundary pixels from the ink regions, always checking that the removal of each pixel does not disconnect any part of the shape or create a hole where none existed. After several passes the thick strokes collapse to single-pixel-wide threads that faithfully trace the path of each line.

From the skeleton I extracted five topological measurements for every sigil. Connected components count how many separate ink islands exist after skeletonization. Junctions are pixels with three or more skeleton neighbors, meaning branch points where strokes diverge. Endpoints are pixels with exactly one neighbor, the tips of lines. Holes are enclosed regions found by counting interior contours. And the Euler number, computed as components minus holes, provides a single integer summarizing the topological complexity of the entire shape.

The trick I had to learn was that neighbor counting on a pixel grid is not as straightforward as it sounds. A pixel has eight neighbors in a square grid, and whether you count diagonal neighbors changes the connectivity model entirely. The choice between four-connected and eight-connected neighborhoods ripples through every topological measurement. I settled on eight-connected because the sigil strokes are drawn at arbitrary angles and a four-connected model would fragment diagonal lines into disconnected segments.

What this revealed was striking variation. Some sigils are single connected components with no holes, essentially tree-shaped networks of branching lines. Others contain multiple disconnected pieces, enclosed loops, and complex multi-junction intersections. The Euler number ranged from negative values, indicating more holes than components, to large positive values for sprawling tree-like designs.

CHAPTER 3

Mapping the Intersections

`03_junction_endpoint_detection.py`

Knowing that a sigil has twelve junctions and eight endpoints tells you something about its complexity, but it tells you nothing about what those junctions and endpoints look like. The third script set out to classify the terminal decorations, the visual flourishes at the tips of each line.

The approach begins at the skeleton. Every endpoint identified in the previous step becomes a seed for local analysis. Around each endpoint I examined a small circular neighborhood, roughly fifteen pixels in radius, and computed several shape descriptors: the ink density within that neighborhood, the aspect ratio of the local ink blob, and its circularity, defined as four pi times area divided by perimeter squared.

From these descriptors a simple classification scheme emerged. Endpoints where the neighborhood contains a compact circular blob with circularity above 0.7 are classified as circle terminals. Those with very high ink density and a cross-like shape are filled or cross terminals. Long narrow regions aligned horizontally become horizontal bars, and the same aligned vertically become vertical bars. Everything else falls into the simple category, a bare line ending with no decoration.

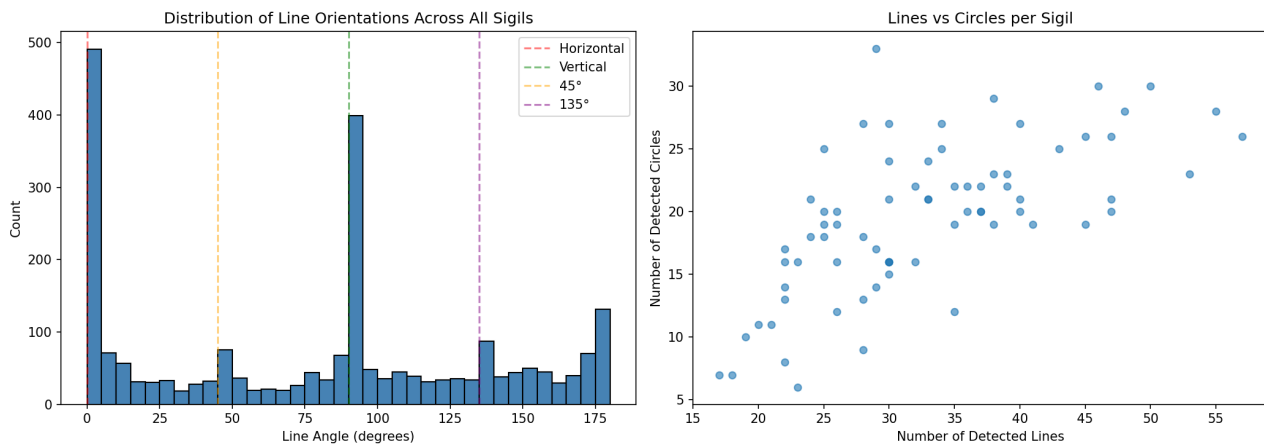
What I learned here was that classification does not require machine learning. A thoughtfully chosen set of geometric measurements combined with sensible thresholds can produce a working taxonomy. The vocabulary that emerged, circles, bars, crosses, and simple endings, maps surprisingly well onto the visual vocabulary that occult scholars have described qualitatively for centuries. The quantitative approach confirmed what they saw by eye and added precision to their descriptions.

The dominant terminal type across the corpus turned out to be the simple bare ending, accounting for roughly eighty percent of all terminals. Circle decorations appeared in about ten percent, and the various bar and cross types made up the remainder. This distribution became one of the inputs to the generative model built later in the pipeline.

CHAPTER 4

Detecting Geometry

04_hough_geometry.py



Line orientation distribution across all 72 sigils, revealing a strong preference for orthogonal angles.

The skeleton captures topology but discards geometry. A junction tells you that three lines meet at a point, but it does not tell you the angles between them or whether any of the strokes are perfectly straight or circular. For that I needed the Hough transform, a classical technique that detects parametric shapes, lines and circles, even when they are fragmented or partially obscured.

The Hough transform works by voting. For line detection, every edge pixel in the image casts votes in a parameter space defined by angle and distance from the origin. Collinear pixels all vote for the same line parameters, producing a peak in the accumulator that reveals the line even if parts of it are missing. The probabilistic variant I used, available as `HoughLinesP` in OpenCV, adds efficiency by operating on random subsets of edge pixels and returning line segments rather than infinite lines.

For each detected line segment I computed its orientation angle, normalized to the range zero to one hundred eighty degrees, and accumulated these angles into a twelve-bin histogram covering fifteen-degree intervals. The result was revelatory. The sigils overwhelmingly favor orthogonal orientations. The zero-degree bin, representing horizontal strokes, contains nearly thirty percent of all detected lines. The ninety-degree bin, representing vertical strokes, holds another twenty percent. Diagonal angles are present but far less common.

Circle detection followed the same Hough principle extended to three parameters: center coordinates and radius. The number of circles per sigil ranged from zero to over sixty, with the high counts typically reflecting concentric structures or clusters of small circular decorations.

What I discovered was that these sigils, despite their mystical provenance, are overwhelmingly built on a rectilinear grid. The angle distribution is not what you would expect from freehand drawing, where orientations would be roughly uniform. It suggests either that the original artist used a straightedge or

that the scribal tradition enforced an implicit grid-based construction method. This finding supported the circuit diagram analogy more strongly than any qualitative observation could.

CHAPTER 5

Measuring Complexity

`05_feature_extraction.py`

By this point I had topology from the skeleton, terminal decorations from the junctions, and geometric measurements from the Hough transform. But I still lacked a unified measure of how complex each sigil is and how its ink is distributed spatially. The fifth script computed a thirty-eight-dimensional feature vector for every sigil, capturing everything from gross complexity to fine spatial balance.

The centerpiece of this feature set is the fractal dimension, estimated using the box-counting method. The idea is elegant: overlay a grid of boxes of size ϵ on the image and count how many boxes contain ink. Then repeat with smaller and smaller boxes. If the log of the count scales linearly with the log of one over ϵ , the slope of that line is the fractal dimension. A straight line has dimension one. A filled square has dimension two. The sigils fell between 0.9 and 1.7, with most clustering around 1.2 to 1.4, confirming that they are more complex than simple lines but far from space-filling.

Bilateral symmetry was measured by flipping the binary image horizontally and vertically and computing the Pearson correlation between the original and flipped versions. A perfectly symmetric image scores one. Most sigils scored between 0.3 and 0.7 on horizontal symmetry and somewhat lower on vertical, suggesting a modest but inconsistent tendency toward balanced designs.

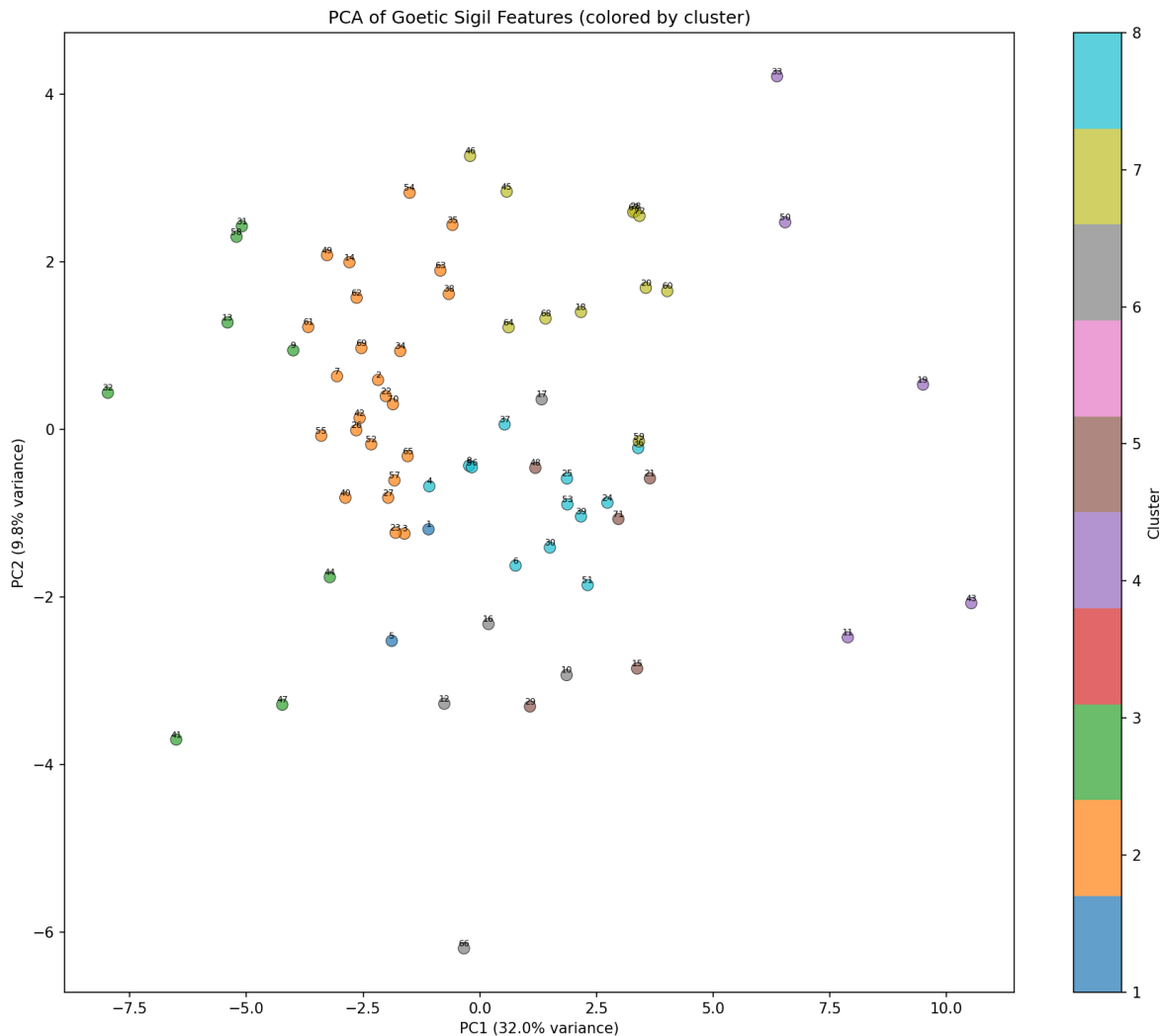
The radial profile divides the image into eight concentric annular rings centered on the centroid and measures the ink density in each ring. This reveals whether a sigil is center-heavy, like a compact knot, or periphery-heavy, with strokes radiating outward. Similarly, quadrant density splits the image into four quadrants and measures the balance among them.

What I had to learn was that feature engineering is an art of deciding what matters. Every measurement is a lens that highlights some aspect of the image while ignoring others. The fractal dimension captures overall complexity but nothing about spatial arrangement. The radial profile captures spatial distribution but nothing about connectivity. The power comes from combining them into a vector that captures the sigil from multiple angles simultaneously, which is exactly what the clustering step would need.

CHAPTER 6

Discovering Families

06_clustering.py



PCA projection of all 72 sigils into two dimensions, colored by cluster assignment. Eight distinct sigil families emerge.

With a thirty-eight-dimensional feature vector for each sigil, the natural question was whether the sigils group into families, clusters of seals that share structural characteristics despite depicting different demons. I used Ward's method for hierarchical agglomerative clustering, a bottom-up approach that starts with each sigil as its own cluster and progressively merges the two most similar clusters at each step.

Ward's method minimizes the increase in total within-cluster variance at each merge. This produces compact, spherical clusters, which suited the data well because there was no reason to expect elongated or irregularly shaped groups. Before clustering I normalized the features using a standard scaler, subtracting the mean and dividing by the standard deviation for each feature. Without this step, features with large ranges like junction counts would dominate the distance calculations while features with small ranges like symmetry scores would be effectively invisible.

The dendrogram, a tree diagram showing the merge history, revealed a natural division into eight clusters when cut at an appropriate height. I chose the cut point by looking for the largest jump in merge distance, which indicates a transition from merging genuinely similar sigils to forcing dissimilar ones together.

To visualize the clusters I projected the thirty-eight-dimensional feature space down to two dimensions using Principal Component Analysis. PCA finds the directions of maximum variance in the data and projects each point onto those directions. The first two principal components captured about forty percent of the total variance, enough to see the cluster structure clearly in a scatter plot.

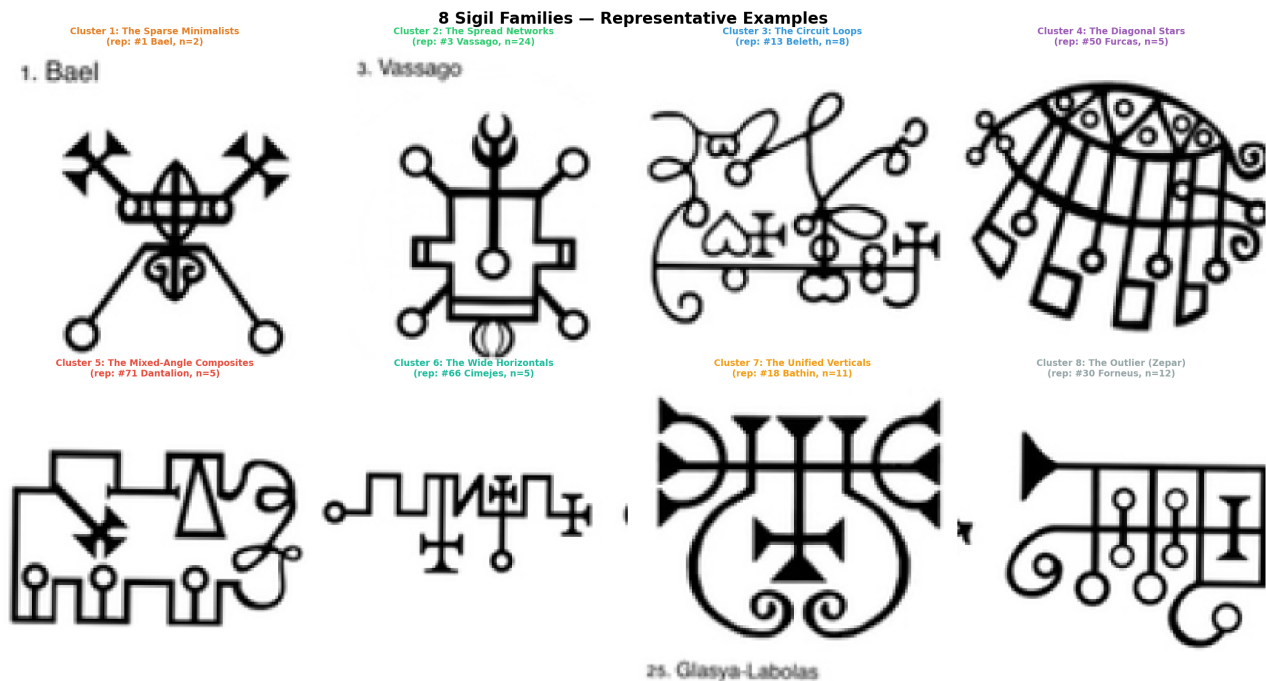
The eight families that emerged had distinctive profiles. The Spread Networks, the largest family with twenty-four members, are moderately complex with antler-like branching. The Circuit Loops contain the most complex sigils, with enclosed loops and high ink density reminiscent of electrical circuits. The Sparse Minimalists are the simplest, with few junctions and low ink coverage. One cluster contained only a single member, Zepar, whose unusual aspect ratio and peripheral ink distribution set it apart from every other sigil in the corpus.

What surprised me was how coherent these families are. They were discovered purely from visual measurements with no knowledge of the demons' ranks, abilities, or position in the Goetia sequence. Yet each family has a visually recognizable style that even a casual observer can learn to identify after seeing a few examples.

CHAPTER 7

Visualizing the Families

07_cluster_composites.py



Overview composite showing representative members and key statistics for each of the eight sigil families.

Numbers and scatter plots describe the families abstractly, but to truly understand them you need to see them. The seventh script assembled composite images for each cluster, tiling the member sigils into a labeled grid alongside their aggregate statistics.

The implementation is straightforward but taught me an important lesson about presentation. A grid of images arranged programmatically in matplotlib using GridSpec must be carefully sized so that each cell is large enough to see detail but the overall figure remains manageable. I computed the grid dimensions from the cluster size, aiming for roughly square layouts, and added titles showing the cluster name, member count, and average feature values.

The composite images proved to be the single most useful output of the entire project for communicating results to non-technical audiences. A scatter plot in PCA space means nothing to someone unfamiliar with dimensionality reduction, but a grid of similar-looking sigils arranged side by side immediately conveys what a family is and why the grouping makes sense.

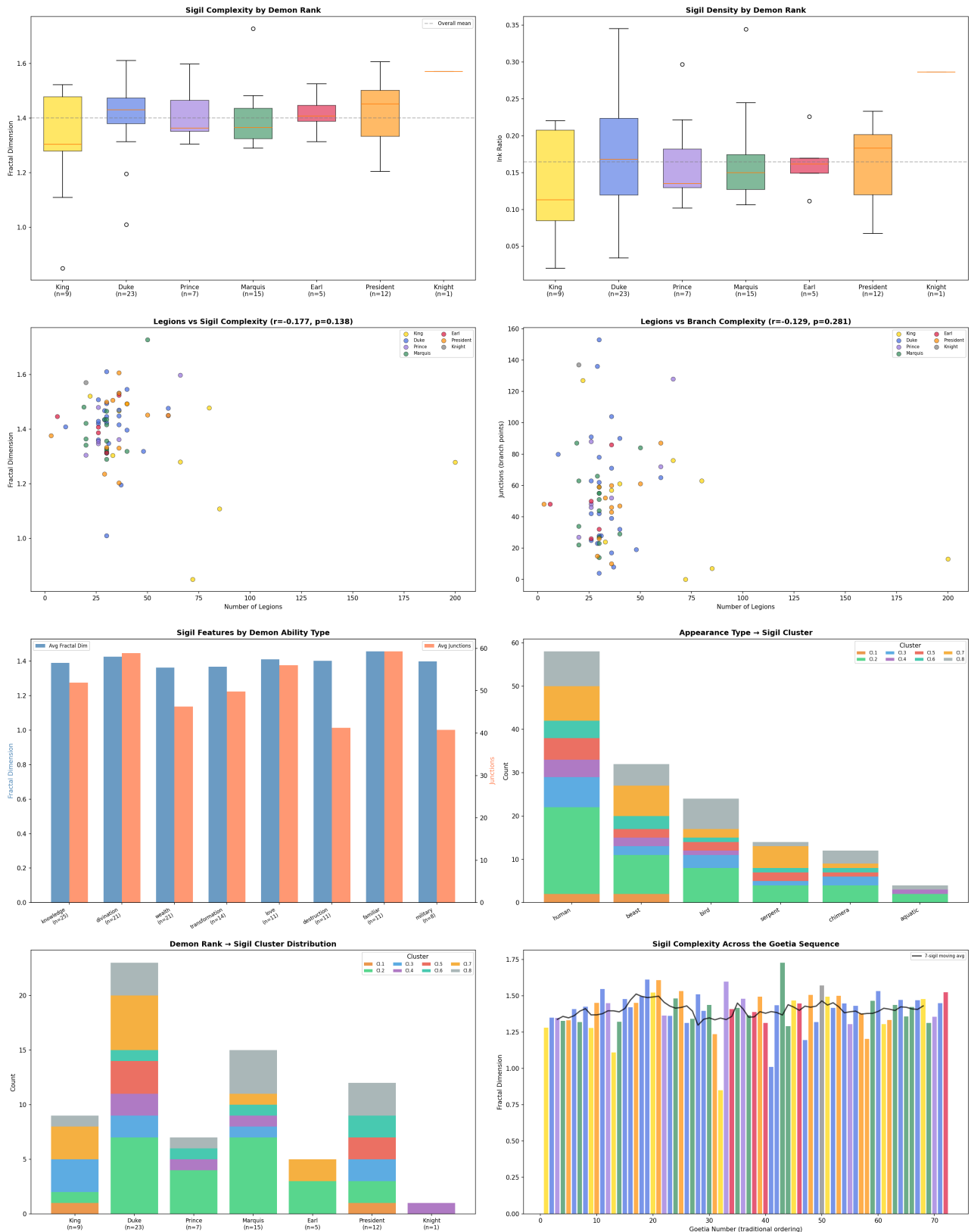
What this step reinforced was that visualization is not a final polish applied after the analysis is complete. It is an integral part of the analytical process. Building the composites revealed cluster assignments that looked questionable on paper but made perfect sense visually, and vice versa. In several cases the composites prompted me to revisit the clustering parameters and try different

numbers of clusters before settling on eight as the most interpretable division.

CHAPTER 8

Bridging Text and Image

08_textual_correlation.py



Statistical tests for correlation between textual attributes (rank, legions, abilities) and visual features of the sigils.

Every demon in the Goetia comes with textual metadata: a rank in the infernal hierarchy such as King or Duke, a count of legions commanded, a list of abilities, and a description of its appearance. The eighth script asked whether any of this textual information predicts the visual structure of the corresponding sigil.

I used a battery of non-parametric statistical tests because the feature distributions are not normal and the sample sizes for some ranks are small. The Kruskal-Wallis test, a non-parametric alternative to one-way ANOVA, checked whether the median feature values differ across ranks. Spearman rank correlation tested for monotonic relationships between legion count and individual features. Chi-squared tests examined whether rank and cluster membership are independent.

The results were definitive and somewhat disappointing from a mystical perspective. There is no statistically significant relationship between a demon's rank and the complexity, symmetry, or any other measured property of its sigil. The p-value for the rank versus fractal dimension comparison was 0.89, about as far from significance as possible. Legion count shows no correlation with any visual feature. The cluster assignments are distributed across ranks almost exactly as chance would predict.

The one suggestive finding involved ability categories. Demons associated with love tend to have slightly more junctions than those associated with knowledge or destruction, but the effect is small and may not survive correction for multiple comparisons.

What this taught me about statistical reasoning is as valuable as any image processing technique. The absence of a relationship is itself a finding. It tells us that whoever designed these sigils did not systematically vary their visual complexity to reflect the demon's properties. The sigils and the text appear to be independent systems, which raises interesting questions about the historical process by which the Goetia was compiled. Were the sigils added later by a different hand? Or did the artist simply not care about matching visual complexity to hierarchical rank?

CHAPTER 9

Building the Database

`09_build_database.py`

Nine scripts had now produced a scattered landscape of JSON files, each containing one slice of the analysis. The ninth script unified them into a single comprehensive database, a master JSON file where every demon has a complete record combining its textual metadata, visual features, topological measurements, geometric properties, cluster assignment, and base64-encoded images.

The base64 encoding step was a key design decision. By embedding the sigil, skeleton, and junction map images directly in the JSON as text strings, the entire database becomes self-contained. The interactive dashboard built later can load a single file and render everything without needing access to the original image directory. This makes the dashboard deployable to GitHub Pages as a truly standalone single-file application.

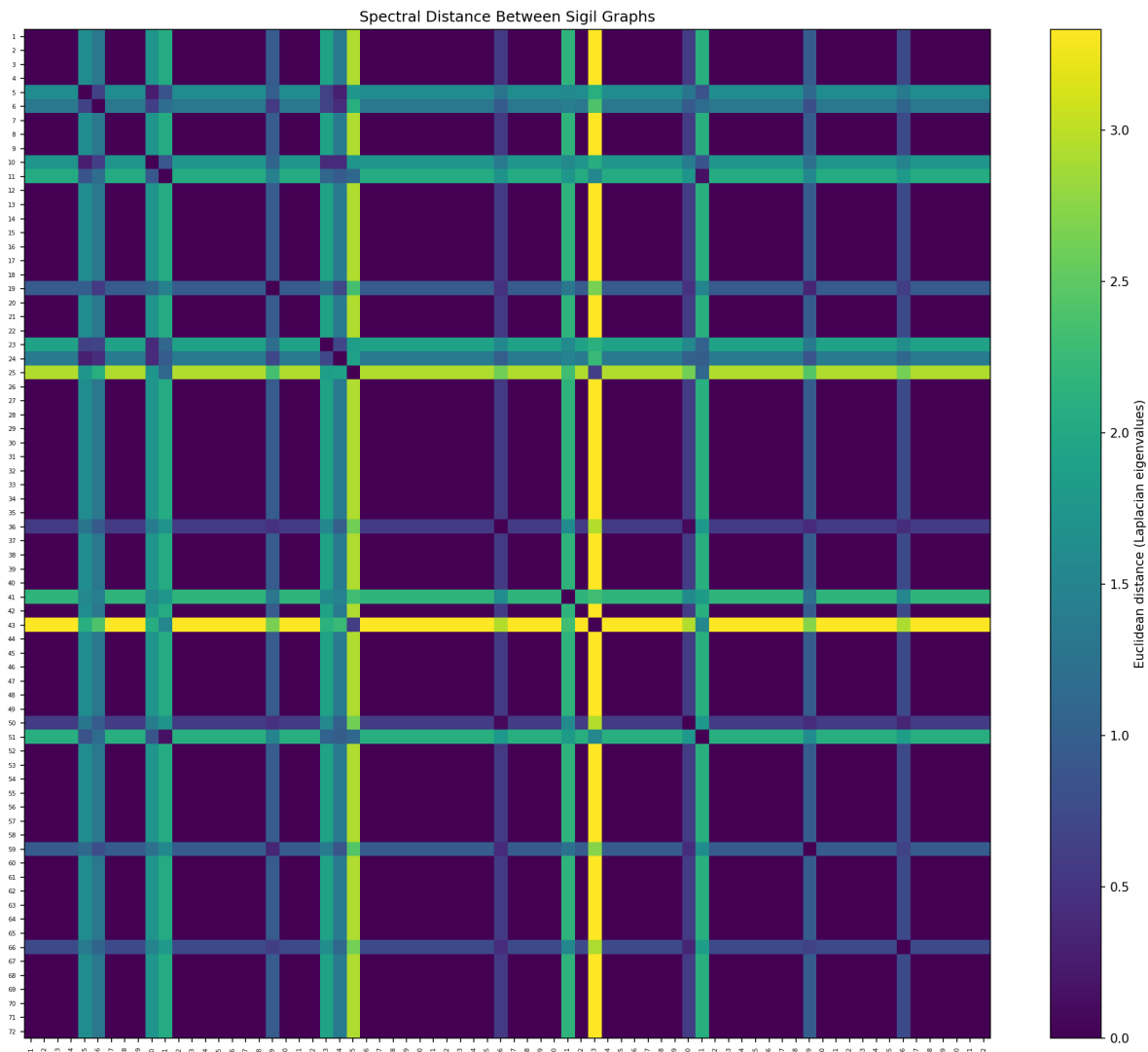
The data integration process was a lesson in schema design. Each source file uses slightly different key names and nesting structures. The build script maps and normalizes these into a consistent schema where every demon record has the same set of top-level keys: `id`, `name`, `rank`, `legions`, `abilities`, `appearance`, `cluster_id`, `cluster_name`, `features`, `topology`, `geometry`, `junction_detail`, and `images`. Missing data receives sensible defaults rather than being omitted, which prevents null reference errors in the downstream dashboard code.

What I took away from this step is that data engineering is the unglamorous backbone of any analysis pipeline. The individual analysis scripts are intellectually interesting, but without a clean integration layer their outputs remain scattered fragments. The unified database is what makes the interactive dashboard possible and what makes the project feel like a coherent whole rather than a collection of disconnected experiments.

CHAPTER 10

Thinking in Graphs

10_graph_theory.py



Spectral distance heatmap showing pairwise similarity between all 72 sigils based on their graph Laplacian eigenvalues.

The skeleton of a sigil is a network. Junctions are nodes and the strokes connecting them are edges. The tenth script made this metaphor literal by converting each skeleton into a NetworkX graph and computing a suite of graph-theoretic metrics.

Building the graph from the skeleton image requires walking the skeleton pixels. Starting from each junction pixel, I traced outward along each branch until reaching another junction or an endpoint, recording the path length in pixels as the edge weight. The result is a weighted undirected graph where node degree tells you how many strokes converge at a junction and edge weight tells you how long each stroke is.

From these graphs I computed classic metrics: average degree, graph diameter (the longest shortest path), clustering coefficient (the tendency for neighbors of a node to also be connected to each other), and betweenness centrality (how often a node lies on the shortest path between other pairs of nodes). The clustering coefficients turned out to be extremely low, averaging around 0.003, confirming that the sigils are almost entirely tree-like structures with very few triangles or cycles.

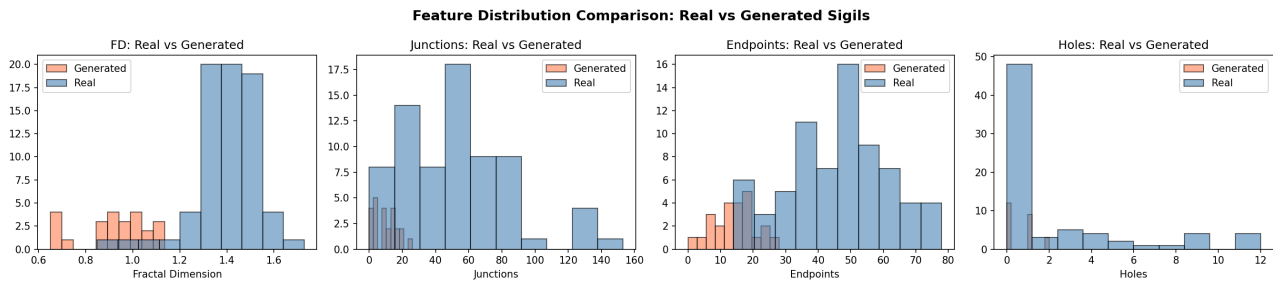
The most powerful analysis came from spectral graph theory. The Laplacian matrix of a graph, defined as the degree matrix minus the adjacency matrix, has eigenvalues that encode the graph's global structure. The first non-zero eigenvalue, called the algebraic connectivity or Fiedler value, measures how well-connected the graph is. The full spectrum of eigenvalues forms a fingerprint that can be compared between graphs using Euclidean distance, providing a principled measure of structural similarity that considers the entire topology at once rather than comparing individual metrics.

What I learned is that graph theory provides a vocabulary for structure that pixel-based features cannot match. Two sigils might have similar fractal dimensions and ink ratios but completely different graph structures, or vice versa. The spectral fingerprint captures a dimension of similarity that complements rather than duplicates the feature vector.

CHAPTER 11

Learning to Generate

11_generative_model.py



Feature distribution comparison between real Goetic sigils and synthetically generated ones.

If you truly understand a visual language, you should be able to speak it. The eleventh script tested this by learning a probabilistic grammar from the corpus of real sigils and using it to generate new ones.

The grammar captures the statistical regularities discovered in previous scripts. It records the probability distribution over line angles, with horizontal and vertical heavily favored. It records the probability of each terminal decoration type, with simple endings dominating at eighty-one percent. It records the probability that a sigil contains enclosed holes, the typical number of junctions, and the distribution of ink density.

Generation proceeds by sampling from these distributions. First, the number of primary strokes is drawn from the observed distribution. Each stroke receives a random angle sampled according to the angle probabilities. Strokes are placed at random positions within a canvas and extended to random lengths drawn from the observed range. At intersections, new branches may sprout with a probability proportional to the observed junction density. Terminal decorations are added to endpoints according to the decoration probabilities. Finally, a small circle may be added near the center with a probability derived from the corpus.

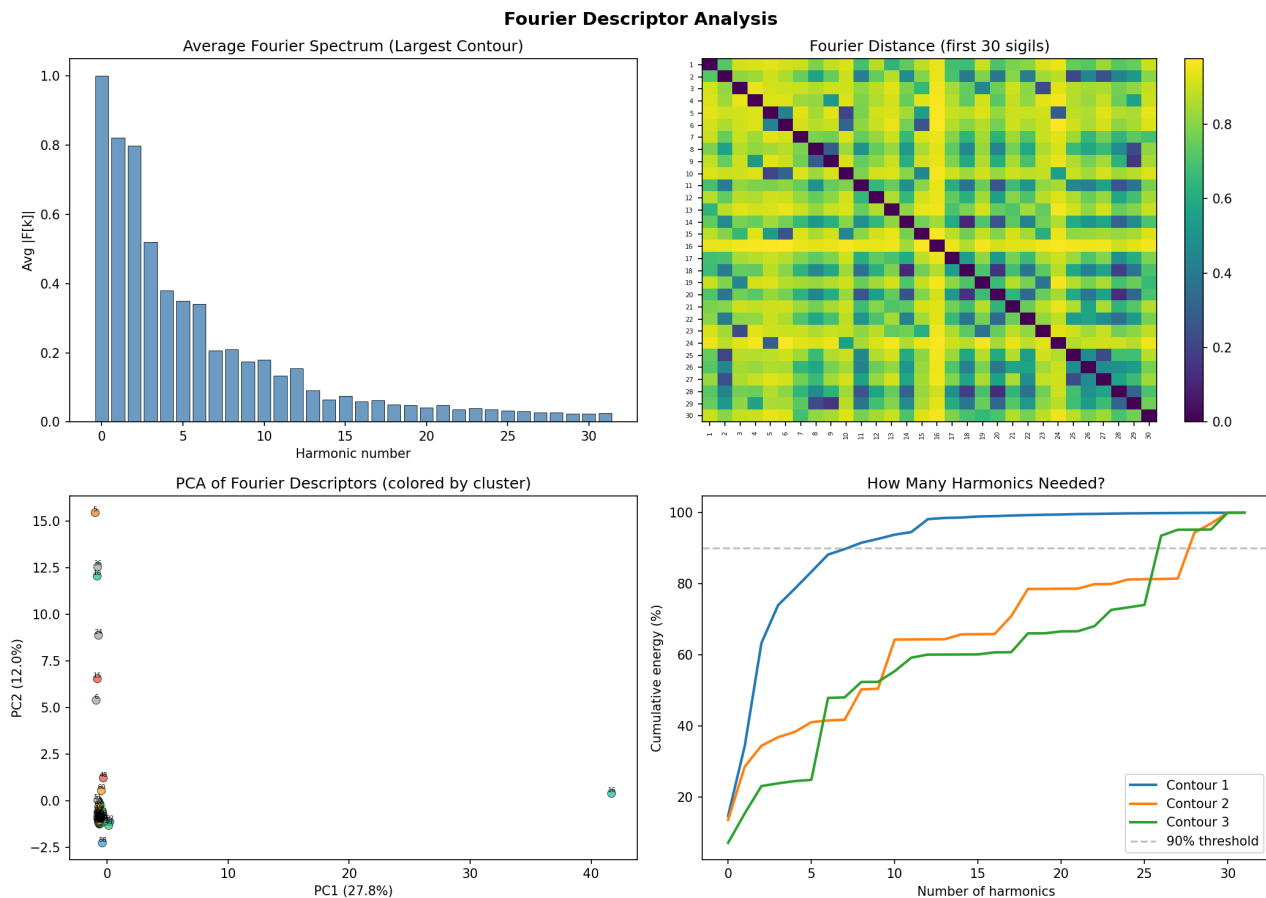
The results are visually recognizable as sigil-like but distinguishable from real ones. A feature comparison showed that the generated sigils have lower junction counts and more uniform spatial distribution than the real corpus. This gap reveals what the grammar fails to capture: the real sigils have a compositional structure, a sense of a central spine with subsidiary branches arranged around it, that a purely statistical model does not encode.

What this exercise taught me is the power and the limits of generative modeling without deep learning. The probabilistic grammar captures marginal statistics perfectly but misses conditional dependencies, the way the placement of one stroke constrains the placement of the next. Capturing those dependencies would require a more sophisticated model, perhaps a graph grammar or a recurrent neural network, which remains a direction for future work.

CHAPTER 12

The Shape of Sound

12_fourier_descriptors.py



Fourier analysis showing harmonic energy distribution and the most similar sigil pairs.

The Fourier transform is usually associated with sound and signal processing, but it works on any periodic or quasi-periodic signal, including the sequence of coordinates that traces a contour. The twelfth script applied Fourier descriptors to the sigil outlines, encoding their shapes in the frequency domain.

The method starts by extracting the largest contours from each binary sigil image. Each contour is a sequence of x-y coordinates describing a closed curve. These coordinates are combined into complex numbers, $x + iy$, forming a complex-valued signal that can be transformed with the discrete Fourier transform. The resulting frequency components are called Fourier descriptors.

The genius of this representation is that it can be made invariant to position, scale, and rotation through simple normalizations. Removing the DC component (the zero-frequency term) eliminates translation. Dividing all coefficients by the magnitude of the first harmonic eliminates scale. Taking the magnitude of each coefficient eliminates rotation. What remains is a pure shape descriptor that does not change if you move, resize, or rotate the sigil.

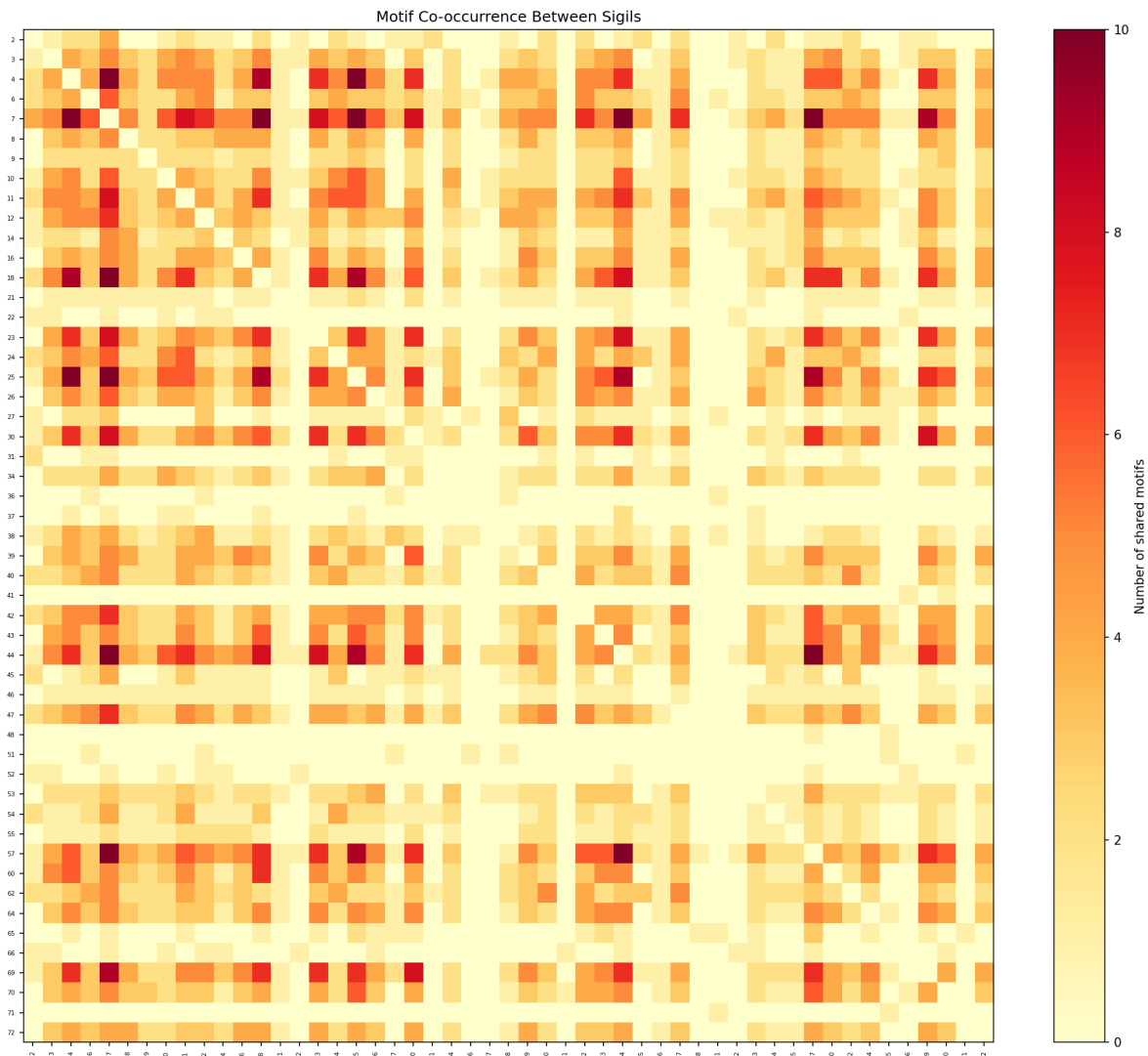
I kept the first thirty-two harmonics for each of the top five contours per sigil, yielding a one-hundred-sixty-dimensional fingerprint. Comparing these fingerprints using cosine distance revealed which sigils have the most similar outlines regardless of their orientation or size. The most similar pair turned out to be Vine and Bifrons, with a cosine distance of just 0.027, confirming a visual resemblance that is apparent to the eye but difficult to quantify without this mathematical framework.

What I found most satisfying about this technique is how it translates a visual question into a signal processing question. Asking whether two shapes are similar becomes asking whether their frequency spectra overlap, which is a well-understood problem with decades of theory behind it. Roughly ninety percent of the shape information is carried in the first eight to ten harmonics, mirroring the observation in audio processing that a sound's timbre is largely determined by its first few overtones.

CHAPTER 13

Shared Vocabulary

13_template_matching.py



Co-occurrence network showing which sigils share the most sub-pattern motifs.

If the sigils were constructed from a finite set of building blocks, those building blocks should recur across multiple seals. The thirteenth script searched for shared sub-patterns, or motifs, using normalized cross-correlation template matching.

The approach works by extracting small patches, twenty-eight and thirty-six pixels square, centered on each junction and endpoint in every sigil's skeleton. Each patch captures the local structure around an interesting point: how many strokes converge, at what angles, and what decorations are present. These patches become templates that are slid across every other sigil's skeleton image, computing the normalized cross-correlation at each position. Where the correlation exceeds a threshold of 0.65, a match is declared.

After deduplication to remove spatially overlapping matches, the script found forty-six unique motifs that appear in two or more sigils. It then built a co-occurrence matrix recording how many motifs each pair of sigils shares. Amon, the seventh demon, turned out to be the most connected node in this network, sharing motifs with twenty other sigils and suggesting that its visual elements are particularly generic or foundational.

The motif co-occurrence network provides a complementary view of similarity to both the feature vector clustering and the Fourier descriptors. Two sigils might have very different overall shapes and feature profiles but share specific junction configurations, the way two buildings might look nothing alike overall but share the same style of doorknob.

What I learned is that template matching, despite being one of the oldest techniques in computer vision, remains powerful when the templates are chosen thoughtfully. The key insight is that not every pixel region is equally interesting. By centering templates on topologically significant points like junctions and endpoints, the search focuses on the most structurally meaningful parts of each sigil rather than wasting effort matching featureless stretches of background.

CHAPTER 14

Reading the Sequence

14_historical_ordering.py



Sequential trend analysis showing how sigil features evolve across the Goetia ordering from demon 1 to demon 72.

The seventy-two demons are numbered in a specific order that has been preserved across manuscripts for centuries. The fourteenth script asked whether the visual properties of the sigils change systematically along this sequence, which might reveal something about the historical process of their creation.

I computed Spearman and Kendall rank correlations between each feature and the Goetia number, testing whether later sigils tend to be more or less complex than earlier ones. The Savitzky-Golay filter, a smoothing technique that fits successive low-degree polynomials to overlapping windows of data, was applied to the feature sequences to reveal trends obscured by point-to-point noise.

Three features showed statistically significant trends. The number of detected lines increases modestly

through the sequence, as does the number of circles and the count of connected components. However, the composite complexity measure, the fractal dimension, shows no trend at all. This suggests that while later sigils tend to incorporate more geometric elements, their overall visual density remains roughly constant. The artist may have been adding more detail to later designs without necessarily making them more complex in the topological sense.

A sliding-window variance analysis revealed that the earliest sigils, roughly numbers one through ten, are the most consistent in their feature values, while the middle of the sequence shows the most variability. This pattern is consistent with a hypothesis that the first few sigils were drawn with particular care or according to a strict template, after which the artist exercised more creative freedom.

The CUSUM chart, a cumulative sum control chart borrowed from industrial quality control, identified the sharpest stylistic transitions in the sequence. These change points did not correspond to rank boundaries or any known textual division, further supporting the conclusion from the textual correlation analysis that the sigils and the text evolved independently.

What I learned from this script is that time-series techniques can be applied to any sequentially ordered data, not just temporal measurements. The Goetia ordering is not a time series in the conventional sense, but treating it as one and applying smoothing, autocorrelation, and change-point detection revealed patterns that would be invisible in a static cross-sectional analysis.

CHAPTER 15

Preparing for Interactivity

`15_precompute_dashboard_data.py`

All of the analysis so far produced static outputs: JSON files, PNG plots, and printed summaries. The fifteenth script prepared the data for an interactive web dashboard by precomputing the most expensive calculations and storing their results in small, fast-loading JSON files.

The heaviest computation was finding nearest neighbors across three different similarity metrics. For each of the seventy-two sigils, the script identified the eight most similar companions according to Fourier descriptor cosine distance, spectral graph distance, and feature vector Euclidean distance. These three neighbor lists provide different perspectives on similarity: Fourier neighbors share outline shapes, spectral neighbors share graph topology, and feature neighbors share aggregate measurements.

The PCA biplot data was precomputed as both point coordinates and feature loading vectors. The loading vectors show which original features contribute most to each principal component, enabling an interactive visualization where hovering over a loading arrow highlights which measurement is pulling a sigil in that direction.

Per-cluster z-score profiles were computed to identify each family's distinguishing features. A z-score above one means the cluster's average for that feature is more than one standard deviation above the global mean, making it a defining characteristic. These profiles power a feature contribution inspector that explains why each sigil was assigned to its cluster.

What this step reinforced is that performance-sensitive web applications require a separation between analysis and presentation. Running PCA or computing pairwise distances on every page load would make the dashboard unusably slow. By precomputing everything and storing the results in lightweight JSON, the browser only needs to parse and render, not calculate. This pattern of offline computation plus online visualization is fundamental to interactive data applications.

CHAPTER 16

The Correction

16_verify_mapping.py, 17_resegment_v2.py, 18_resegment_v3.py

Sixteen scripts into the project, a critical error surfaced. The original segmentation script had merged three adjacent sigils into a single image at position sixteen, shifting every subsequent mapping by one. Demon number sixteen, Zepar, was actually showing the combined seals of Zepar, Botis, and Bathin fused together, and every seal from seventeen onward was attributed to the wrong demon.

The verification script confirmed the problem by analyzing the aspect ratios of all extracted regions. Three entries had aspect ratios above two, meaning they were more than twice as wide as they were tall, a clear sign of merged images. The fix required abandoning the morphological dilation approach entirely in favor of a fixed grid-based segmentation.

The original image is laid out in eight rows of ten columns, with specific positions left empty or containing variant labels. Knowing this structure, the second version of the segmentation script divided the image using predetermined pixel boundaries for each column and row edge. This brute-force approach is less elegant than automatic contour detection but is immune to the merging problem because it never asks adjacent cells to determine their own boundaries.

The third version refined the extraction further by including each demon's correct English name label, which appears as text printed directly above each seal in the source image. Previous versions had stripped all text to isolate just the seal drawing, but including the name provides valuable context and makes the extracted images more self-documenting. The challenge was distinguishing the correct name above the seal from the wrong name that bleeds in from the row below. This required scanning the ink density row by row from the bottom of each cell, identifying a gap of three or more consecutive blank rows, and cutting everything below that gap.

What this episode taught me is that upstream errors propagate silently through an entire pipeline. Every script from two through fifteen had produced plausible-looking results with the wrong data, and none of them raised an alarm. The statistical properties of the corpus barely changed after the correction, which means the error could have gone undetected indefinitely. Verification against ground truth, not just internal consistency, is essential for any analysis that begins with data extraction.

Conclusion

This project began as an experiment in applying computational tools to an unusual corpus and became a sustained exercise in learning by doing. Eighteen scripts, each solving a specific problem, collectively demonstrate a progression from basic image processing through topology, geometry, statistical inference, graph theory, signal processing, and generative modeling.

The technical skills practiced include OpenCV for computer vision, scikit-image for skeletonization, SciPy for clustering and statistical testing, NetworkX for graph analysis, NumPy for numerical computing, and matplotlib for visualization. Beyond individual tools, the project taught me how to design a multi-stage analysis pipeline, how to engineer features that capture meaningful variation, how to evaluate whether observed patterns are statistically significant, and how to build interactive web dashboards for exploring complex datasets.

The findings about the sigils themselves are genuinely interesting. The seals are not arbitrary scribbles but structured designs built predominantly on an orthogonal grid, employing a finite vocabulary of terminal decorations, and falling into eight recognizable families based on their topological and geometric properties. The text and the images appear to be independent systems, assembled separately rather than designed as a unified whole. And the sequence shows subtle trends in geometric detail even as overall complexity remains stable.

Whether these findings illuminate the historical origins of the Goetia or merely reflect the constraints of medieval draftsmanship is a question for scholars of Western esotericism. What they certainly demonstrate is that computational methods can reveal structure in visual corpora that resists qualitative analysis alone, and that the tools of modern data science are far more general than the domains in which they were originally developed.